TRACE: A Unified and Lightweight Data Engine for Multi-Modal Discrete Spatiotemporal Data Management and Analytics

Ziyi Liu
Department of CSE, HKUST
zyl@ust.hk

Wenli Sun, Jiajia Li
Shenyang Aerospace University
{sunwenli,lijiajia}@stu.sau.edu.cn

Zhoujin Tian, Jiawei Li
Department of CSE, HKUST
{ztianaf,jlijz}@connect.ust.hk

Chrysanthi Kosyfaki
Department of CSE, HKUST
@ust.hk

Lei Li
DSA Thrust, HKUST(GZ) and HKUST
thorli@ust.hk

Xiaofang Zhou
Department of CSE, HKUST
zxf@cse.ust.hk

Abstract

Effectively managing extensive discrete spatiotemporal data is crucial for applications such as autonomous driving, smart city infrastructures, and digital twins. Conventional Gist/R-tree-based indexing approaches encounter scalability issues and incur significant update costs when processing multimodal queries that include trajectory data, point clouds, and mesh data. This paper presents a lightweight and unified data engine designed for the management and analysis of discrete spatiotemporal data, termed as TRACE. It employs incremental index construction, separates storage and computation, and incorporates query-focused optimization strategies. It uses partitioning based on spatial, temporal, and feature-based criteria to minimize indexing complexity and enhance query efficiency. Furthermore, The engine integrates an incremental KD-tree with Morton-coded hierarchical grids, facilitating rapid nearest-neighbor and topological queries. Additionally, a meta-index supports cross-partition queries and dynamic updates. Experimental evaluations reveal that TRACE delivers superior query latency and scalability compared to leading methods, establishing it as an efficient high-performance option for managing large-scale spatiotemporal data.

PVLDB Reference Format:
Ziyi Liu, Wenli Sun, Jiajia Li, Zhoujin Tian, Jiawei Li, Chrysanthi Kosyfaki, Lei Li, and Xiaofang Zhou. TRACE: A Unified and Lightweight Data Engine for Multi-Modal Discrete Spatiotemporal Data Management and Analytics. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 Introduction

Modern sensing and mapping technologies have led to an explosion of Discrete SpatioTemporal (DST) data that describe the physical world[2, 25]. These data appear in three dominant modalities: point clouds, trajectories, and meshes. Point clouds capture dense geometric detail from LiDAR or depth sensors [10, 26]; trajectories encode object motion over time [22]; and meshes represent surface structures using topological connectivity [13]. Together, they support critical applications such as auto-driving [23], digital twin construction [9], and urban simulation [17]

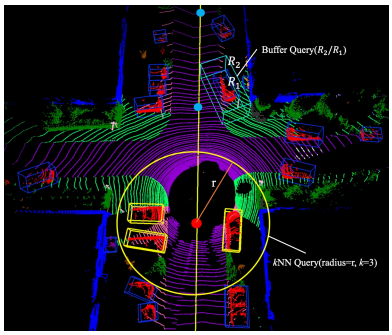


Figure 1: Example of Cross-Modal Queries in DST

Although each modality describes different properties of the physical world, they are frequently used together in practice. For example, self-driving cars process point clouds to perceive surroundings, analyze trajectories to plan motion, and use meshes to understand road geometry [7]. However, existing systems [4, 14, 20, 24]treat each modality in isolation: they rely on separate storage, indexing, and query engines, each with its own interface and data model. As a result, cross-modal queries require issuing subqueries over multiple indexes, materializing large intermediate results, and performing expensive application-level joins [5, 8]. These fragmented access patterns lead to redundancy, high latency, and limited scalability[21].

Figure 1 illustrates two representative cross-modal queries commonly encountered in autonomous driving and digital twin platforms. The first is a kNN query: given a vehicle's current position (from a trajectory point), find the k nearest point cloud objects within a surrounding radius (e.g., for hazard detection). The second is a buffer query: given two nested regions R_1 and R_2 defined by mesh boundaries (e.g., lanes), retrieve all trajectory and point cloud data within R_2/R_1, such as for collision detection or rule violation analysis. In traditional systems, each query must be executed in stages. For the kNN query, the trajectory engine locates the vehicle, the point cloud engine retrieves nearby objects, and the mesh

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

engine checks spatial constraints, each using its own index and loading data independently.

This separation causes three major inefficiencies: 1) Identical spatial filters are applied in each engine separately. This leads to repeated index traversal, overlapping region scans, and excessive I/O, even when the same region is filtered multiple times. 2) Without joint pruning, each engine materializes large partial results. For example, the point cloud engine may return all nearby objects, many of which are irrelevant once trajectory and mesh context is applied. These results must be sorted, joined, and filtered at the application level, wasting time and memory. 3) Indexes are often built in memory per modality, consuming excessive RAM and I/O during construction. For example, a kNN query in a city-scale system may require loading and indexing billions of point cloud objects across distributed servers. Without coordination, each server performs local index builds and filtering, followed by global merging. This leads to high resource consumption and query latency, which becomes unacceptable in real-time applications such as live traffic monitoring.

To address these limitations, we present TRACE, a unified data engine for managing and querying TRAjectories, point Clouds, and mEshes. It is designed to support efficient, scalable, and low-latency operations across diverse *DST* data within a single system.

Designing a unified system for DST data raises several fundamental challenges. These stem from the heterogeneity of data structures, the diversity of query patterns, and the need for both scalability and real-time responsiveness. Existing indexing frameworks [2] fall short in supporting efficient integration and incremental maintenance across modalities.

Unified Data Modeling. Point clouds, trajectories, and meshes differ significantly in structure and semantics. Point clouds are unordered spatial samples, trajectories are temporally ordered sequences, and meshes encode topological connectivity. A naïve solution is to store them in separate subsystems, each with its own data layout and query logic. However, this approach makes cross-modal analysis difficult. Data from different modalities must be queried separately and then joined externally, often with mismatched spatial boundaries and redundant disk I/O[1]. For example, to retrieve nearby trajectory and point cloud points, one must scan each index independently and manually join them by location, often materializing large intermediate results and triggering multiple disk accesses for the same region.

An alternative, seen in NoSQL systems like MongoDB or Cassandra, is to flatten all modalities into a unified schema. For example, a wide row might include fields such as timestamp, rgb, speed, adjacency, but most fields are unused in most records, for example, meshes lack timestamps, trajectories have no adjacency. This leads to sparse documents and high storage overhead, especially in row-based systems where nulls and field names are repeatedly stored. Columnar formats (e.g., Parquet) mitigate this but assume rigid schemas and offer limited support for spatial indexing or streaming updates.

To address this, TRACE introduces a compact, polymorphic point-level data model. All records share a minimal set of core attributes, location and optional timestamp, while modality-specific fields are stored only when needed. Each record carries a small tag indicating its type, enabling efficient filtering and operator dispatch without inflating the schema. This model avoids null-heavy rows, supports heterogeneous fields natively, and enables unified indexing and querying across modalities.

Unified Indexing Challenges for Multi-modal Queries Designing a unified index that works for DST data is not straightforward. These data types support different query patterns: point clouds often use spatial *k*NN, trajectories use time-based range scans, and meshes require access to topological relationships. Supporting all of them in a single index requires careful design to avoid performance bottlenecks. We list the challenges as follows.

Access Patterns. Point clouds are spatial and unordered, so *k*NN queries depend on fast spatial filtering. Trajectories are time-ordered, so queries often focus on time intervals. Meshes rely on vertex connectivity, which is a different kind of access. If we build separate indexes for each type, it becomes difficult to run cross-modal queries, like 'find all trajectory points and point cloud samples near this building in the last 10 minutes'. These queries need to access multiple indexes, scan each one, and then join the results. This wastes time and memory. TRACE solves this by allowing all data types to be indexed and queried together in a shared spatial-temporal structure.

Disk Access. If each modality has its own index, the same geographic area can be stored in multiple disk blocks. When a cross-modal query touches this region, it loads the same region multiple times from different indexes. This causes redundant disk I/O and prevents efficient prefetching. TRACE avoids this by using a global spatial partitioning index shared by all modalities. Each spatial block is read only once and is used for all types of data inside it.

Limited Resources. DST datasets can span billions of records and terabytes of storage, while deployment environments (e.g., edge devices, embedded systems) may have limited RAM and disk bandwidth. In-memory indexing and bulk-loading approaches quickly become infeasible. TRACE adopts an out-of-core storage model. Only relevant data chunks are loaded at query time, guided by lightweight voxel summaries. This minimizes memory usage and disk I/O, allowing the system to scale to large datasets even on constrained hardware.

Incremental Updates. In dynamic environments such as autonomous driving or continuous 3D mapping, data are generated at high frequency. Traditional indexing structures, optimized for static data, struggle to support fast insertions, deletions, or index maintenance without complete rebuilds. TRACE supports incremental updates within localized OK-tree partitions. Insertions and deletions are applied in place, while subtree rebalancing is deferred or performed in the background. This design enables responsive query performance even under high-frequency streaming updates.

In summary, our main contributions are as follows.

- We design a unified data engine that manages point clouds, trajectories, and meshes while avoiding query processing through multiple indexing models to improve the efficiency of cross-modality queries.
- We propose voxel-based masks separate raw-data access from index operations, which results in substantially reduced *I/O* loads during high concurrency scenarios.
- We provide an incremental method that avoids expensive global reorganizations while serving fast-changing data

processes in fields such as autonomous driving and real-time mapping.
- We show *TRACE* outperforms SOTAs for indexing and both single- and cross-modality queries on large-scale real and synthetic datasets.

2 Preliminaries

2.1 DST Data and UST Record

We first define the three base data types and then introduce the proposed unified data type.

2.1.1 ***DST Data.*** We focus on the following three *DST* data:

1) Point clouds is a large set of *3D* points, where each point records spatial coordinates and additional attributes such as intensity or color. It offers high-precision geometric details and is widely used in autonomous driving and urban mapping. Common operations include *range* and *kNN* queries and clustering for object detection and segmentation. For example, one-stage detectors [10, 27] directly predict object boundaries and classes from raw point clouds, while two-stage approaches [19, 28] first generate region proposals before refining predictions.

2) Trajectories are time-ordered sequences of spatial points recording the movement of objects. The inherent temporal order supports queries like time-constrained range queries and *kNN* searches for moving objects, as well as data mining tasks [29] like trajectory clustering and anomaly detection. Indexes like the *TB-tree* [15], *TPR-tree* [18], and *TrajTree* [16], along with preprocessing techniques such as segmentation, synchronization, and map-matching, have been developed to facilitate efficient trajectory analytics.

3) Meshes are 3D structured polygons derived from point clouds through surface reconstruction [11, 12] to represent objects. Mesh queries typically focus on surface extraction, curvature analysis, and geometric matching. They capture both geometric and topological information, making them ideal for visualization, simulation, and detailed *3D* modeling. Compared to raw point clouds, meshes offer a more compact and continuous representation of surfaces.

2.1.2 ***UST Record.*** Although many systems have supported each one of these data types [5, 6] separately, a unified framework that supports cross-modality indexing, querying, and mining remains lacking. To fill this gap, we propose the *Unified SpatioTemporal (UST)* data type to model the above three *DST* data consistently and support integrated access across modalities. Specifically, let S° denote the raw multimodal dataset with three disjoint subsets $S^\circ = S_{\text{Tra}} \cup S_{\text{C}} \cup S_{\text{E}}$, where S_{Tra}, S_{C}, and S_{E} denote the sets of trajectory data, point cloud samples, and mesh vertices. A decoupling pipeline FD transforms S° into a flattened record set $P = \text{FD}(S^\circ) = \{d_1, .., d_n\}$, where $n \leq |S^\circ|$. Each $d_i \in P$ is a self-contained semantic unit equipped with the following spatiotemporal and structural information:

1) $\underline{\texttt{rid}_i}$: A unique system-level record identifier.

2) $\underline{\texttt{d.ID}} \in \mathbb{S}_{\text{obj}}$: A symbolic identifier representing the source-level data unit where d originates. Identifiers are drawn from a finite set $\mathbb{S}_{\text{obj}}$, with values $\text{Tra}_i \subseteq S_{Tra}$, $\text{C}_i \subseteq S_C$, or $\text{E}_i \subseteq S_E$. Therefore, when a query only returns partial data points of a cluster, the system can still reconstruct the complete cluster using d.ID. For meshes containing point adjacency relationships, we maintain a adjacency table whose ID is associated with d.ID. For example, the record $\{\texttt{d.ID}, \{\texttt{rid}_1, \texttt{rid}_2, \texttt{rid}_3\}, \{\texttt{rid}_1, \texttt{rid}_2, \texttt{rid}_4\}\}$ indicates that the Mesh associated with d.ID contains a triangular face formed by vertices $\texttt{rid}_1$, $\texttt{rid}_2$, and $\texttt{rid}_3$, and another triangular face formed by vertices $\texttt{rid}_1$, $\texttt{rid}_2$, and $\texttt{rid}_4$.

3) $\underline{\texttt{d.STP}}$: SpatioTemporal Position of the point:

– d.STP[location] = $(d_x, d_y, d_z) \in \mathbb{R}^3$: 3D spatial coordinates of the record in a global reference frame.

– d.STP[time] = $d_t \in \mathbb{T} \cup \{\texttt{NULL}\}$: Timestamp since epoch, or NULL for static sources like mesh vertices of buildings.

4) $\underline{\texttt{d.domain[d.bit]}} \in \{\texttt{0b001}, \texttt{0b010}, \texttt{0b100}\}$ encodes record modalities with a fixed-length bitmask, where 0b001, 0b010, and 0b100 represent trajectory points, point cloud samples, and mesh vertices. It enables efficient modality-based filtering using bitwise operations, as well as flexible polymorphic operator dispatch during query processing. For example, the system can quickly extract all point cloud records by evaluating whether the second bit is set. This design also enables hybrid querying. When the query bitmask contains multiple active bits, it returns results of different data types in a single query, thereby supporting cross-modal analysis and unified index reuse.

5) $\underline{\texttt{d.user}}$: A string identifier of the user, agent, or sensor that collected or generated the data point. It offers provenance-awareness, enabling queries to filter or aggregate data based on its source. This feature is valuable in collaborative sensing or privacy-aware systems, where queries might focus on user-specific data (e.g., retrieve all samples from Sensor-A in the past hour).

Together, this schema ensures *UST* is compact, expressive, and query-efficient across modalities and spatiotemporal workloads.

2.2 TRACE Queries

TRACE supports a broad spectrum of spatiotemporal queries through a unified indexing framework built on the *UST* data model. Unlike existing systems that require separate indexes and result merging across modalities, TRACE enables seamless execution of both single- and cross-modality queries with minimal overhead.

The representative queries are listed as follows.

Q_1***: DST 3D Polygonal Range Query***: Given a time interval $[t_1, t_2]$ and a 3D spatial range R, Q_1 is defined as $q(R, t_1, t_2) = \{d \in S | d.time \in [t_1, t_2] \wedge d.location \in R\}$. It returns all data points that lie within the 3D range R during the specified time interval.

The *3D* range R is specified by a base polygon on a slanted *3D* plane and a vertical height (d_x, d_y, d_z), creating a column. This base is a planar area in the *3D* space, defined by custom *3D* vertices or connected mesh faces on the same plane.

Q_2***: DST Buffer Query***: Given a 3D range R, time interval $[t_1, t_2]$, buffer radius r, the query $q(P, r, t_1, t_2)$ returns all data points within a hollow cylindrical shell formed by expanding the boundary of R by r, subtracting the inner cylinder R and applying the given temporal constraints.

Queries Q_1 and Q_2 apply spatiotemporal constraints over geometric regions and support multimodal access through predicate filtering on d.bit. TRACE further enhances these queries by supporting flexible *3D* polygonal range queries, where spatial regions are defined by arbitrarily oriented polygonal bases combined with vertical extent. This allows for precise region filtering in scenarios

such as robotics or simulation environments involving complex spatial boundaries.

Q_3***: DST*** k***NN Query*** Given a query point d° and an integer k, the query $q(d^\circ, k)$ returns the k closest points $\{d_1, \ldots, d_k\} \subseteq S$ to d°, such that for any $p \in S \setminus \{d_1, \ldots, d_k\}$, $\text{dist}(d_i, d^\circ) \leq \text{dist}(p, d^\circ), \forall i \in [1, k]$ where distance is typically Euclidean (i.e., L_2 norm).

Q_3 operate either within a single modality or across multiple modalities. For example, TRACE can identify the nearest points exclusively within point cloud data or jointly across point cloud and trajectory records in a single index traversal.

Q_4***: User-Specific Query*** Given a user ID U and a time interval $[t_1, t_2]$, the query $q(U, t_1, t_2) = \{d \in S | d.time \in [t_1, t_2] \wedge \texttt{d.user} = U\}$ returns all points associated with U during the given time.

Q_4 support filtering in all data types and are evaluated together with spatial and temporal predicates.

Q_5***: DST Similarity Query*** Given a query object Q, a candidate set $\varphi = \{T_1, \ldots, T_N\}$ within $[t_1, t_2]$, a similarity function M, and an integer k, the query $q(Q, \varphi, M, k)$ returns a subset $S \subseteq \varphi$, where $|S| = k$, such that for any $T \in S$ and $T' \in \varphi \setminus S$, $D(Q, T) < D(Q, T')$

Q_5 evaluate structural similarity among objects such as trajectories, point cloud clusters, or mesh patches, and are constrained to a single modality by requiring that all involved points satisfy `d.bit = TYPE_MASK`. In addition to point-wise comparisons, TRACE supports similarity queries over sets of points, allowing cluster-to-cluster matching in point clouds or group-level trajectory comparisons. This provides stronger expressiveness for shape-based or behavior-aware retrieval under non-uniform data densities.

These query capabilities are supported by a unified index that combines an *out-of-core Octree* and an *incremental Kd-tree (iKd-tree)*. This architecture eliminates the need for multiple per-modality index structures and enables efficient predicate pushdown and early pruning across large-scale multimodal datasets.

2.3 Problem Formulation

In this paper, we focus on efficient and unified processing of spatiotemporal queries over multimodal DST data, including point clouds, trajectories, and meshes. These data are periodically collected in large volumes and contain complex spatial, temporal, and semantic information. We aim to support diverse query types from Q_1-Q_5 over this data while addressing the challenges posed by scale, heterogeneity, and real-time requirements.

Input: Let $S^\circ = S_{\text{Tra}} \cup S_{\text{C}} \cup S_{\text{E}}$ be a multimodal dataset consisting of trajectories, point clouds, and mesh vertices. The dataset is transformed via a decoupling pipeline FD into a unified record set $P = \{d_1, ..., d_n\}$, where each record d_i conforms to the UST schema.

Query Class: Let q be a query from a class $\mathbb{Q}$ of spatiotemporal queries supported by *TRACE*, including Q_1-Q_6. Each query operates on the unified dataset P and returns a result A, formally denoted as: $q : P \rightarrow A$, where $A \subseteq P$.

Problem Statement: Given a multimodal data set S° transformed into a unified record set $P = \texttt{FD}(S^\circ)$ and a query $q \in \mathbb{Q}$, the goal is to efficiently evaluate q using a lightweight, unified and dynamic index, under constrained memory and I/O resources.

3 Related Work

Extensive research efforts have been devoted to developing database systems and their extensions for managing spatial and spatiotemporal data. In this section, we briefly review existing systems and discuss their capabilities and limitations in handling three-dimensional discrete spatiotemporal data.

3.1 Spatial Relational Database System

To effectively manage spatial data within traditional databases, various commercial and open-source solutions have been developed. Prominent commercial products include *IBM DB2 Spatial Extender*, *Microsoft SQL Severl Spatial*, and *Oracle Spatial*. *Oracle Spatial* offers robust spatial indexing and diverse query capabilities for points, lines, polygons, and basic *3D* geometries, including point clouds. Open-source systems such as *PostGIS* built upon *PostgreSQL*, *MySQL*, and *SpatiaLite* built upon *SQLite*, also offer substantial spatial query functionality. In particular, *PostGIS* stands out because of its extensive support for various spatial types and fundamental trajectory processing capabilities.

Although these spatial *RDBMS* have been widely adopted and are successful in managing static and *2D* spatial data, their performance deteriorates when large-scale *3D* discrete spatiotemporal data are handed over. For instance, spatial indexing methods such as GIST/R-trees used in these systems often suffer from excessive bouding-box overlaps in *3D* scenarios, leading to increased query overhead and inefficient incremental updates. Moreover, their limited support for temporal trajectory and complex scene-oriented data significantly restricts their applicability to emerging applications, such as autonomous driving, *unmanned aerial vehicle(UAV)* routing, and digital twins of urban environments.

3.2 Spatial DBMS Extensions

To overcome scalability challenges in managing large-scale spatial data, various distributed spatial frameworks have emerged. *SpatialHadoop* extends *Apache Hadoop*'s *MapReduce* paradigm with spatial indexing mechanisms suitable for ditributed processing of large spatial datasets. *GeoSpark*, built on distributed data stores such as *Apache Accumulo* and *HBase*, implements distributed indexing structures optimized for scalability. *Hadoop-GIS* and *STARK* also provide spatiotemporal query capabilities in distributed environments using Hadoop and MapReduce paradigms. *LocationSpark* extends Spark by employing specialized in-memory spatial indexing techniques, achieving efficient query processing with good fault tolerance. However, despite the advantages of scalability, these distributed frameworks remain primarily tailored towards simpler *2D* spatial queries or limited spatiotemporal analytics. Their indexing strategies are often variants or extensions of traditional spatial indexes, not fully optimized for the complexity and scale of three-dimensional discrete spatiotemporal data. Moreover, cross-modality queries that involve combinations of trajectories, point clouds, and meshes remain inefficient or unsupported due to the fragmented indexing approach.

Several specialized extensions target trajectory management or *3D* data handling separately. *MobilityDB* and *Pg-Trajectory* extend *PostgreSQL* and *PostGIS* to natively support trajectories, implementing dedicated temporal indexing structures and spatiotemporal

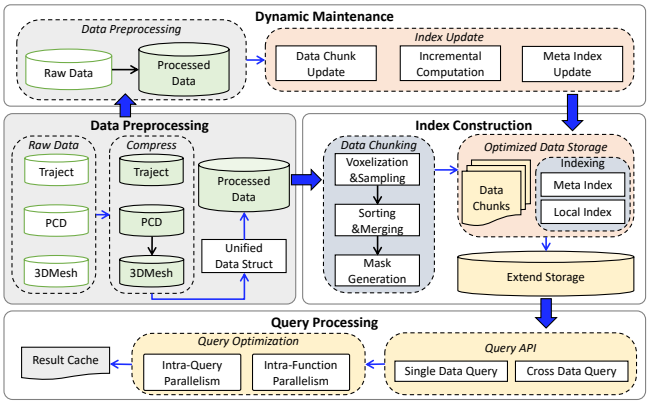


Figure 2: The Architecture of TRACE

operations. Other systems like *3DCityDB* leverage *PostGIS* or *Oracle Spatial* to manage structured *3D* city models and *Building Information Modeling (BIM)* data. However, these solutions typically focus on specific single modalities, leading to fragmented database management environments. This fragmentation further complicates efficient cross-modality query processing, which is critical for holistic analysis in real-world applications such as collision detection, real-time planning, and city-scale simulation. Current spatial *RDBMS* and their extensions offer strong capabilities for traditional spatial data, yet struggle to manage large-scale three-dimensional spatiotemporal data involving multiple modalities, incremental updates, and real-time complex queries. Our work addresses these limitations by proposing a unified indexing framework designed specifically to handle discrete spatiotemporal data, incorporating efficient incremental updates, flexible cross-modality querying, and optimized storage and computation strategies suitable for resource-constrained environments, bridging the gap left by existing solutions.

4 TRACE System Architecture

The physical implementation of TRACE follows the master-worker architecture. The master node schedules tasks, while the worker nodes handle data storage and computation tasks.

As illustrated in Figure 2, each worker has the following four modules that process the *UST* data in pipeline:

1) Data Preprocessing. To support modality-independent indexing and querying, all incoming trajectory, point cloud, and mesh data are transformed into UST records via a standardized decoding pipeline. This transformation includes spatial alignment into a shared global coordinate system, timestamp normalization, and encoding into a compact binary layout with modality-aware bitmasks. Domain-specific fields (e.g., trajectory speed, point cloud intensity) are organized under a shared `d.domain[]` container, while shared spatiotemporal fields are promoted to top-level attributes. This standardization ensures consistent memory layout, indexing semantics, and operator dispatch, laying the foundation for unified downstream processing.

2) Index Construction. To support scalable and efficient querying over large-scale UST datasets, TRACE adopts a hierarchical indexing architecture. At the top level, a global meta-index partitions the dataset into coarse-grained spatiotemporal regions using voxel-based density masks. This enables early pruning of irrelevant blocks and supports parallel processing across partitions. Within each partition, TRACE builds a two-layer local index to support fine-grained access and modality-specific operations. The first layer adaptively subdivides the space based on local data distribution, while the second layer organizes records by modality and enables efficient lookup within each cell. This layered design ensures that spatial filtering, temporal selection, and modality-specific constraints can be jointly evaluated, minimizing I/O overhead and supporting cross-modal execution. The detailed indexing structure is introduced in Section 5.1.

3) Dynamic Index Maintenance. TRACE supports efficient ingestion of streaming or incrementally updated data. New records are first transformed into UST format and assigned to target partitions via the global meta-index. Within each partition, local indexes are updated in place using lazy update policies that avoid full reprocessing. Metadata structures, including partition summaries and the global meta-index, are asynchronously updated to reflect the changes while maintaining consistency. This design ensures low-latency updates, minimizes locking overhead, and supports real-time queryability over evolving multimodal datasets.

4) Query Processing. The Query Processing framework in TRACE integrates three key components: the Query API, Query Optimization, and Result Cache, together enabling efficient access over large-scale multimodal spatiotemporal data. The Query API provides unified support for both Single Data Queries (e.g., trajectory-based range or kNN search) and Cross Data Queries that span multiple data types. TRACE determines the execution path based on the `d.domain[d.bit]` attribute, leveraging a shared index structure for all queries. The Query Optimization layer enhances performance through two levels of parallelism. Intra-Query Parallelism splits a query into independent sub-tasks over spatial partitions, allowing concurrent execution across partitions. Intra-Function Parallelism further accelerates compute-intensive operations such as geometric filtering or similarity scoring by parallelizing them within each sub-task. The Result Cache stores outputs from previous queries to avoid redundant computation for repeated or overlapping requests. This lightweight mechanism improves responsiveness in exploratory or interactive query workloads.

5 Unified Data Engine

In this section, we present the details of the unified data model and scalable indexing pipeline of TRACE. Firstly, we introduce the data and index structure in Section 5.1. Then we describe how to load different individual datasets in Section 5. Finally, we describe how to maintain TRACE incrementally in Section 5.2.4.

5.1 TRACE Index Structure

TRACE employs a hierarchical indexing architecture tailored for scalable and low-latency access to large-scale spatiotemporal data. As shown in Figure 3, the system organizes data using a global meta-index and localized composite structures called *OK-trees*, each

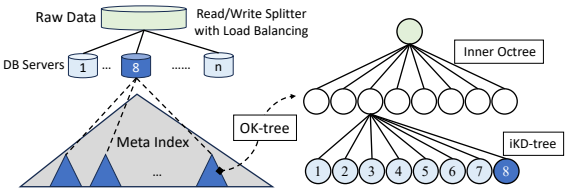


Figure 3: A Conceptual Illustration of the Hierarchical Index

combining a lightweight spatial octree with an incremental kd-tree(ikd-tree).

At the global level, the meta-index is implemented as an out-of-core octree, constructed using a voxel-counting mask (see §4.3). Unlike traditional in-memory trees, this octree is built in a single pass over Morton-ordered data, using only sequential disk scans and fixed-size voxel aggregations. It avoids recursive memory construction and retains only a lightweight routing structure in memory, while the full tree hierarchy and chunk metadata are stored on disk. Each leaf node corresponds to a spatial chunk and records: its bounding box, byte offset and length in the data file, time range, and UST record count. This enables coarse-grained partition pruning, fast disk seeking, and memory-efficient global coordination. Since the meta-index is read-only after construction and small in size, it remains fully memory-resident during query execution.

Each chunk indexed by the meta-index is internally organized using an OK-tree, which combines a spatial octree with a cell-local incremental kd-tree (iKd-tree). We define the structure as follows:

Definition 5.1 (OK-tree). Let D be the UST record set within a chunk C. An OK-tree consists of: (i) an outer octree that partitions the spatial domain of C based on a configurable density threshold, and (ii) for each octree leaf, a local incremental kd-tree (iKd-tree) constructed over sampled records, indexed by the spatiotemporal key (x, y, z, t).

This composite design preserves spatial coherence and bounds the size of local indexes, enabling efficient cell-level access, filtering, and update. Octree partitioning reduces query scope, while iKd-trees support fine-grained operations such as kNN, range, and similarity queries.

The iKd-tree follows a buffered maintenance model inspired by prior work on dynamic indexing [3]. For each cell, sampled records form a static tree backbone, while new insertions are appended to an update buffer and deletions are marked as tombstones. A full rebuild is triggered only when the number of unmerged records exceeds a predefined threshold τ. All index nodes are stored in offset-addressed binary arrays, supporting memory-mapped loading and SIMD-accelerated traversal.

To optimize query performance and reduce memory usage, index components are serialized into compact layouts. The meta-index and routing metadata are retained in memory, while OK-trees are stored on disk along with their associated data blocks. During query execution, only the necessary OK-trees are loaded, and all partition-level subqueries can be processed independently and in parallel.

This hierarchical indexing design enables TRACE to achieve low-latency query performance and efficient real-time updates at scale, supporting billions of multimodal spatiotemporal records with high concurrency and minimal overhead.

5.2 TRACE Index Construction

The preprocessing pipeline consists of five key stages: (1) voxelization and proportional sampling to balance index granularity and data volume; (2) spatial sorting and merging to improve data locality; (3) hierarchical density summarization via *VC-masks*; (4) top-down *out-of-core Octree* construction for scalable meta-indexing; and (5) per-chunk *OK-tree* construction and optimized data ingestion. This staged design minimizes *I/O* overhead and prepares the dataset for scalable, low-latency querying.

5.2.1 ***Voxelization.*** The preprocessing pipeline begins with voxelizing the global spatial domain into uniform cubic cells. Each raw input record is normalized into a unified binary format, resulting in a consistent *UST* record. Each UST record is assigned to a voxel based on its spatial coordinates.

Object-level Sampling is conducted during the normalization processing when the raw data is being loaded. The primary motivation for applying sampling during voxelization lies in optimizing the performance and practicality of local kNN queries and iKd-tree construction. When constructing iKd trees on full-resolution point cloud data, especially in dense, planar regions such as road surfaces, the index size can grow significantly, resulting in excessive memory consumption and long construction times. Moreover, the results of kNN queries in such regions often consist of spatially close but semantically redundant points, such as those lying on the same flat surface. These neighbors, while geometrically valid, offer limited utility for tasks that require object-level or structural differentiation.

To mitigate these issues, we construct iKd-trees over sampled subsets of the data. The sampling process selects a fixed proportion of points from each object, as identified by `d.ID`, thereby reducing redundancy while preserving the overall spatial distribution of the object. This strategy substantially reduces both the index construction time and memory footprint. More importantly, it improves the semantic relevance of kNN results by increasing inter-object diversity, making it more likely that neighbors correspond to distinct and meaningful objects rather than merely dense clusters of surface points. This approach aligns with TRACE's overarching design goal of supporting lightweight, localized indexing structures that maintain both efficiency and semantic fidelity.

It is important to note that sampling does not affect the construction of the global meta-index. The meta-index is derived from the full dataset via a VC-mask, which aggregates voxel-level point counts independent of sampling. This design reflects a compute–storage separation principle: the out-of-core Octree used for meta-indexing is built solely based on the full density distribution, ensuring accurate and fine-grained global partitioning.

Coarse Spatial Sorting and Merging is applied after voxel-level sampling. This step aims to enhance data locality and prepare for spatially aligned chunking. Each UST record is assigned a *Morton code* (also known as Z-order value) derived from its spatial coordinates. Morton encoding preserves the proximity in a one-dimensional key space, making it suitable for external sorting and hierarchical indexing.

To efficiently sort large datasets that cannot fit entirely into memory, we adopt a two-phase process. In the first phase, the dataset is partitioned into small batches, each sorted in-memory by Morton code. These sorted runs are flushed to disk along with auxiliary maps that associate Morton cell IDs with local record offsets. In the second phase, these batches are merged into a single, globally ordered data file using a multi-pass external merge sort. During merging, auxiliary Morton maps are also merged and updated accordingly.

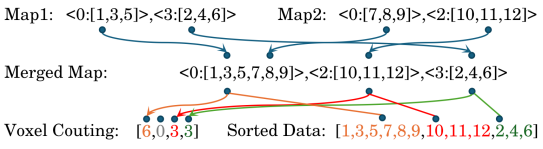


Figure 4: Merging Sorted Files

Example 5.2. In Figure 4, the two parts inside the angle brackets represent <zorder-id: a set of record id>, respectively. We assume that the Z-ordering is defined over a 2×2 cell grid, and that after performing batch-wise sorting on the sampled files under this ordering, two sorted files are produced. The merging process proceeds in increasing order of the zorder of each record. After multiple rounds of merging, the data is stored in a uniquely ordered file. This file can be further split into a voxel-counting file corresponding to each zorder-id, and a sorted data file.

The final output consists of two components: (1) a sorted data file where spatially close records are stored contiguously on disk; and (2) a voxel-counting array derived from the merged Morton maps, indicating the number of records per spatial cell. This spatial ordering significantly improves cache coherence and enables subsequent Octree and VC-mask construction to operate on spatially aligned blocks with minimal random I/O.

5.2.2 ***Meta Index Construction***. Following spatial sorting and sampling, TRACE constructs a global meta-index using a top-down, out-of-core *Octree*. The construction is driven by a compact hierarchical density summary *VC-mask*,which defined in the following.

Definition 5.3 (VC-Mask). Given a spatial dataset sorted along the Z-order curve, the *VC-mask* is a hierarchical summary of voxel counts constructed as follows: 1) The spatial domain is discretized into uniform 3D voxels. 2) A single-pass sequential scan computes the number of records falling into each voxel, forming the base level of the *VC-mask*. 3) Higher levels are generated by aggregating voxel counts hierarchically along the Z-order curve, producing multi-resolution density summaries.

The *VC-mask* compactly represents data density across multiple spatial resolutions and serves as the guide for adaptive top-down Octree partitioning.

Theorem 5.4 (Partitioning Guarantee of VC-mask). *The VC-mask guarantees the following properties for top-down Octree construction:*

(1) Single-pass computability: The base level of the VC-mask is computed in a single sequential pass over the sorted dataset.

(2) Adaptive partitioning: Hierarchical aggregation of voxel counts ensures that the Octree adaptively subdivides dense regions to capture fine spatial detail, while stopping early in sparse regions, so that partition granularity naturally follows data density without over-partitioning.

Meta Index. TRACE constructs a top-down, out-of-core *Octree* as its meta-index, guided by the *VC-mask*. The process avoids loading raw data and operates directly on disk-resident byte offsets. The construction proceeds as follows: First, the dataset is globally sorted along the Z-order curve during preprocessing, and the *VC-mask* is generated in the same order. This ensures that spatial regions align with contiguous disk ranges. Next, the *VC-mask* is traversed top-down to determine which regions exceed a density threshold. Only high-density regions are recursively subdivided, while sparse regions terminate early. Then, for each region selected for subdivision, its corresponding disk byte range is computed and split without accessing the underlying data. Since both the data and mask follow the Z-order sequence, this slicing proceeds via sequential offset calculations. Finally, all subdivision tasks are processed in parallel. Each thread operates independently on metadata and offset ranges, with no data loading or synchronization required. This enables efficient construction with low memory overhead. The resulting *Octree* adapts to data distribution, avoids global data movement, and produces spatially aligned chunks that support fast sequential query access

Example 5.5. Figure 5 illustrates how the *VC-mask* supports efficient out-of-core tree partitioning and query access. Parts (a)–(b) show the bottom-up construction of the *VC-mask*. Starting from fine-grained voxel counts in part (a), higher levels are computed by aggregating voxel values along the Z-order curve. For example, in 2D, every four adjacent voxels are merged to form one coarser-level voxel. The result of such aggregation is shown in part (b), with the top-right region currently being merged. Part (c) demonstrates how the *VC-mask* is used in a top-down manner to guide tree partitioning. High-density regions are recursively subdivided, while sparse areas are skipped. The data, stored in Z-order layout, is aligned with the resulting quadtree leaves. Finally, part (d) illustrates sequential data access during querying. The *VC-mask* indicates how much data to retrieve from each spatial chunk, enabling efficient I/O without scanning irrelevant regions.

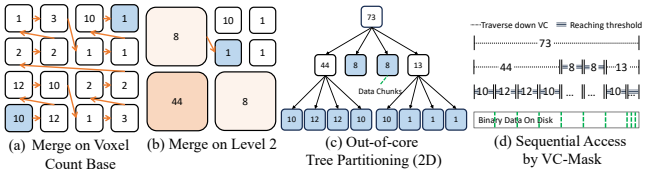


Figure 5: Example of Meta Index Construction

5.2.3 ***Local Index Construction***. After the global *Octree* partitions the dataset into spatial chunks, each chunk is processed independently to build a localized *OK-tree*. The size of each chunk is bounded to fit in memory, enabling in-memory indexing without further disk access. These chunks are also spatially coherent and

aligned with the disk layout, which improves access locality and supports fast sequential reads when needed.

Each *OK-tree* adopts the same top-down construction strategy as the global Octree, recursively organizing data within the chunk into finer spatial subdivisions.

To support point-level queries, each *OK-tree* embeds a dynamic *iKd-tree* constructed over the pre-sampled data within the chunk. This structure is lightweight and built on demand during chunk processing. When combined with `d.ID`-based filtering, it enables fast and accurate *kNN* querying with minimal overhead.

Together, the global *Octree* enables scalable, out-of-core partitioning, while local *OK-tree*s and *iKd-tree*s support efficient in-memory query execution and fast local update. This two-level indexing strategy allows TRACE to maintain both scalability and responsiveness on large spatiotemporal datasets.

Data Ingestion and Storage Layout. After index construction, the dataset is written to disk following a two-level spatial layout: records are first ordered by the leaf nodes of the global *Octree*, and then by the leaf nodes of each chunk's local *OK-tree*. This layout aligns with the *VC-mask* structure and preserves spatial locality across and within chunks, enabling efficient pruning and largely sequential disk access during query execution. Sampled data used for dynamic *iKd-tree* indexing is stored separately for each chunk to support fast *kNN* queries without accessing the full data block.

5.2.4 ***Index Maintenance.*** The update process involves adding new files (`.ply`, `.obj`, or `.csv`) to TRACE and deleting the files by their IDs. The system first locates the target data block using the index structure, then modifies the block, and updates the *VC-mask* for the entire dataset. For deletions, the corresponding sample points are removed, while for insertions, new points are evaluated using both the sampling rate and the sample *VC* to determine their inclusion in the sample set. Finally, both the *ikd-tree* and *Octree* spatial data structures are updated to reflect these changes.

TRACE supports incremental maintenance for both index structures and data blocks, reducing update overhead while preserving query efficiency. Its design follows three key principles:

(1) *Lazy Maintenance*: Each *iKd-tree* is built in memory over sampled data and maintained using a lazy update strategy. Deletions are marked with flags, and insertions are appended without immediate rebalancing. Since each tree is bounded in size and localized to a chunk, a full rebuild is triggered only when the number of updates exceeds a threshold.
(2) *Localized Rebuild*: Each *Octree* node tracks its data volume and update count. When updates in a subtree exceed a specified ratio, that branch is selectively rebuilt in the background. This localized strategy minimizes global disruption and ensures continuous query availability.
(3) *Deferred Compaction*: Data blocks follow the same lazy strategy. Insertions are appended, and deletions are marked in place. Once the number of modifications crosses a threshold, a background compaction phase removes invalid entries and restores block order, maintaining both storage efficiency and sequential I/O performance.

6 Query Processing

TRACE supports a comprehensive set of spatiotemporal queries, designed to exploit the system's hierarchical indexing framework comprising a global meta-index and localized OK-tree structures. Each query is executed in two stages: (i) coarse partition pruning via the meta-index, and (ii) fine-grained data retrieval through incremental *iKd-tree*s and optimized temporal filtering.

Q1 retrieves data points located within a specified polygonal region, bounded vertically by a height range, and temporally by a given interval. The query begins by leveraging the meta-index to identify partitions that intersect the polygonal region. Within each relevant partition, spatial filtering is performed by evaluating whether the location of each data point falls within the polygon and the height range. Temporal constraints are applied using binary search on the sorted timestamps of the filtered records. This query type is fully resolved in memory, without requiring access to *iKd-tree*s.

Q2 identifies data points located within a buffer zone surrounding a defined polygonal boundary. The query process starts by expanding the spatial region based on the specified buffer radius. The meta-index filters partitions that overlap the buffered region. Local refinement is achieved by evaluating the inclusion of data points based on distance within the buffer area. As with the previous queries, temporal constraints are enforced via binary search. *iKd-tree* access is optional, used only when high-precision proximity checks are needed.

Q3 finds the k closest data points to a given query point in both space and time. This query utilizes a priority queue to traverse the out-of-core *Octree*, focusing first on partitions nearest to the query point based on the proximity of the *Z-order*. Within each selected partition, *iKd-tree*s conduct localized *kNN* searches. The resulting candidates are merged into a global priority queue, from which the top k nearest points are selected. Temporal filtering is applied during distance evaluation to ensure that both spatial and temporal proximity are considered.

Q4 retrieves all data points associated with a specific user identifier over a defined time interval. This query bypasses spatial indexing and directly filters data based on user *d.user*. The system uses binary search on the sorted timestamps to apply temporal constraints efficiently. Since spatial location is not relevant, this query avoids the overhead of *Octree* or *iKd-tree* traversal, resulting in fast and direct data access.

Q5 identifies the top-k data objects most similar to a reference object within a specified spatiotemporal range. The query begins with a spatial range scan using the global *Octree* to retrieve candidate objects. These candidates are then grouped according to their data modality (e.g., trajectories or meshes), as defined by the `d.bit` field. A domain-specific similarity metric is computed between each candidate and the reference object. The top-k most similar objects are ranked and returned. Only single-modality data are considered in this query, ensuring consistency in the similarity computation.

7 Parallel Execution

TRACE employs a multi-level parallelization strategy to enhance the performance of both index construction and query execution. By leveraging concurrency across clusters, within data processing

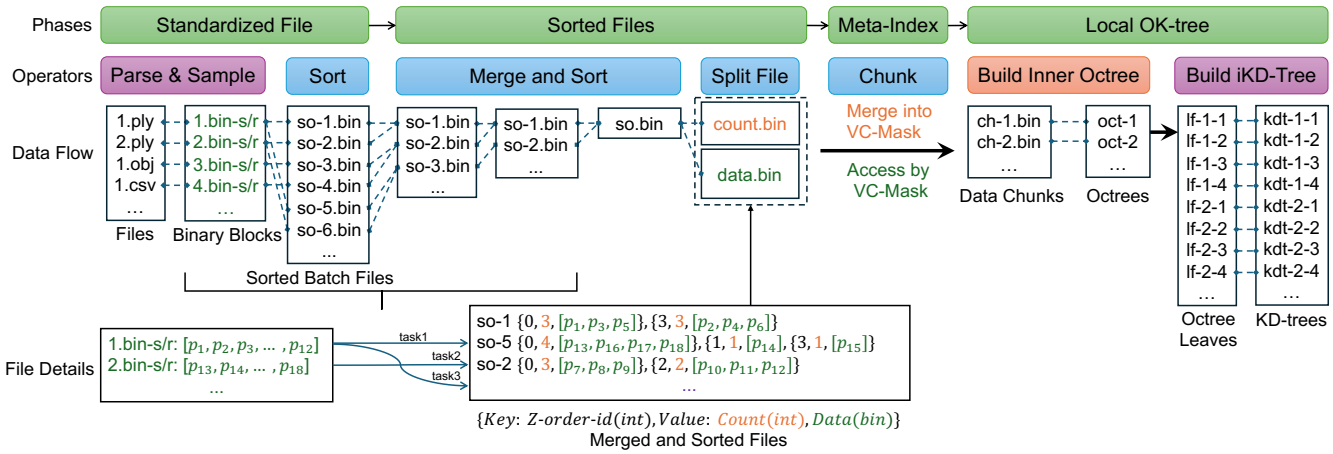


Figure 6: Multi-Threads Processing in Index Construction

pipelines, and at query runtime, TRACE achieves scalable performance on large spatiotemporal datasets.

7.1 Cluster-Level Parallelism

At the cluster level, data ingestion, index construction, and query processing are distributed across worker nodes under the coordination of a master node. For range queries and spatial filters, the master node identifies the worker nodes whose partitions intersect with the query region and dispatches tasks to them in parallel. Each worker processes its assigned data independently, returning partial results that are aggregated in the master. For *kNN* and similarity queries, each worker performs local computations concurrently, and the master node merges intermediate results to compute the final nearest neighbors or similarity rankings.

7.2 Parallelism in Index Construction

The index construction pipeline in TRACE is optimized for fine-grained parallelism, enabling efficient processing of large-scale datasets even on resource-constrained nodes.

The process starts with unified normalization, where each raw data file is parsed and converted to *BLOB* format in parallel threads. Parsing threads also compute local *MBR*s, which are later combined to determine the global spatial extent. By parallelizing parsing across files, this step overlaps data reading with in-memory processing, improving *CPU* utilization and maintaining high disk throughput.

Next, sampling and pre-counting runs in parallel to estimate spatial density and generate the *VC-mask*. Each thread independently samples data from its assigned block, computes *Morton* codes for coarse spatial ordering, and contributes to the global density estimation. This design ensures linear scalability with data size and guides the following top-down *Octree* construction.

After sampling, spatial sorting and merging is performed. Each thread first sorts its local data block by *Morton* code. The sorted segments are then merged through a multi-threaded merge-sort, producing a globally ordered file with spatially continuous data layout. This improves data locality for downstream index construction and query access.

Guided by the *VC-mask*, top-down out-of-core *Octree* construction recursively partitions the data into spatial chunks. Each subdivision task runs in a separate thread, computing partition bounds, *Morton* code ranges, and chunk statistics. This approach avoids over-partitioning in sparse regions and allows controlling tree depth explicitly, unlike bottom-up methods where depth depends on data distribution. Threads write intermediate results to disk, followed by a single-pass assembly of the final meta-index.

Within each spatial chunk, localized *OK-tree* building is performed. Threads build *OK-trees* independently, leveraging the spatial independence of chunks. Out-of-core data access is managed with thread-local buffers, enabling efficient *OK-tree* construction with smooth *I/O* and consistent throughput.

Finally, during the assignment of the original full dataset to the index, multiple binary files can be loaded in parallel and partitioned according to the preconstructed chunk-level *Octree*. Since multiple tasks may attempt to write to the same chunk concurrently, access control is required. Subsequently, the full data within each chunk can be further distributed in parallel to the leaf nodes based on the chunk's internal *Octree*, resulting in the formation of data blocks at the leaves.

7.3 Parallel in querying

TRACE employs multi-level parallelism to achieve high-throughput, low-latency spatiotemporal query execution over large-scale, multimodal datasets. Its query engine maximizes parallelism at both partition and intra-partition levels, while balancing *I/O* and *CPU* workloads to address the diverse performance bottlenecks of spatial queries.

A query starts with meta-index routing, where the global meta-index filters out irrelevant partitions and selects candidate data chunks. This step runs in parallel across worker nodes or thread-local contexts. For spatial queries such as range, polygon, or buffer, the system retrieves relevant *Octree* leaf nodes and groups them into

balanced batches. This improves cache performance and reduces overhead during processing. Then, *I/O*-optimized retrieval divides data access tasks across threads. Each thread reads small data blocks sequentially, balancing disk usage and improving throughput. After data is loaded, *CPU*-optimized filtering partitions tasks such as spatial filtering and geometric intersection across threads. This reduces synchronization and fully uses multi-core *CPU*s.

For similarity queries, TRACE uses a two-step approach: parallel candidate selection (via range queries), followed by per-thread similarity scoring. Each thread reconstructs candidate objects (e.g., trajectories, mesh segments), scores them, and reports top-k results to a global reducer.

Meanwhile, incremental updates are processed asynchronously. Insertions and deletions update local data blocks first. When changes exceed a threshold, background threads rebuild affected *iKd-tree* or *Octree* branches.

8 Experimental Evaluation

In this section, we present an experimental assessment focusing on the performance and scalability of TRACE. Our experiments cover data processing, index construction, and data retrieval tasks. Comparisons with baseline metrics indicate that the optimizations afforded by TRACE produce notable improvements.

Experimental Setup: All experiments were conducted on a Ubuntu server equipped with 4 Intel® Xeon® Gold 6218 CPUs (totaling 160 threads) and 1.5 TB RAM. To better align with real-world industrial environments, we used Docker to simulate a controlled execution environment for index construction and data retrieval. For single-node experiments, we deployed Docker containers, each configured with 4 CPU cores and 16GB RAM, to create a constrained and reproducible testing environment. For distributed environment testing, we simulated a cluster setup on the same physical server by deploying multiple Docker containers. We built two types of container image: The master node is responsible for coordination and scheduling, worker nodes are handling computation and storage. The cluster consisted of one Master node, ten Worker nodes, and a custom Docker network for inter-container communication. Each container was allocated 4 CPU cores and 16GB RAM, ensuring consistency across all nodes. All system modules and algorithms were implemented in C++.

Datasets: To evaluate the performance of TRACE, we employ real-world and synthetic datasets, each consisting of point clouds, trajectories, and mesh objects. The data formats follow common standards: point cloud data is stored in `.PLY` format, recording each point's 3D coordinates, intensity, and timestamp; trajectory data is stored in `.CSV` format, where each trajectory consists of multiple 3D points annotated with timestamps and velocity information; mesh data is stored in `.OBJ` format, containing both vertex and face definitions, where each vertex has 3D coordinates and RGB color values, and each face is represented as a triangle connecting three vertex indices. The proportion between point cloud, trajectory, and mesh data is maintained at approximately 10:1:1 across all datasets.

We used three real-world datasets, including *TST*, *Mongkok*, and *Whampoa*, to evaluate TRACE under realistic urban scale conditions. All point cloud data are sampled from urban scenes in Hong Kong, while the associated trajectory and mesh data are synthetically generated within the same spatial regions to simulate plausible object movements and surface geometry.

- *TST* contains about 6 million point cloud points sampled from the Tsim Sha Tsui area. A total of `1500` trajectories are generated, each trajectory consisting of an average of `550` points. In addition, `4` mesh objects are created, where each mesh contains on average `154k` vertices and `115k` triangular faces.
- *Mongkok* includes 0.5b point cloud points collected from the Mongkok area, generated `40k` trajectories (average `550` nodes per trajectory), and created `84` mesh objects (each with an average of `160k` points and `116k` faces).
- *Whampoa* comprises 0.6b point cloud points, `46k` trajectories (average `550` points), and `84` mesh objects (average `160k` points and `116k` faces per mesh).

To assess TRACE's scalability and robustness under large-scale and controlled conditions, we further construct two synthetic datasets:

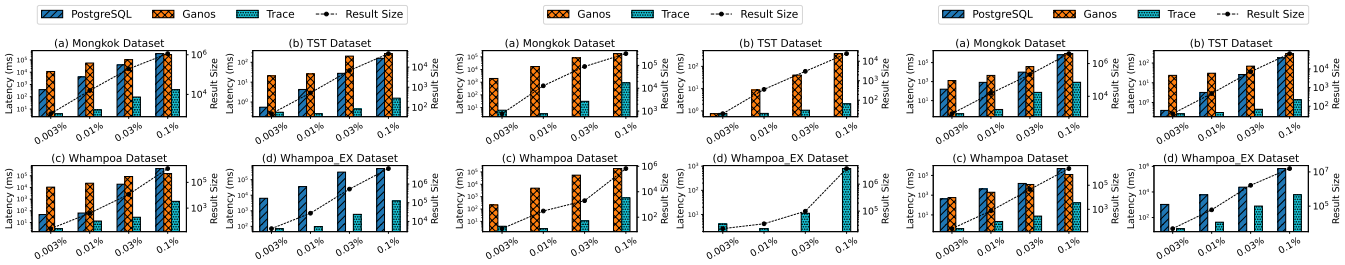
- *Whampoa_EX* is an extended version of the original Whampoa dataset. It is designed to evaluate TRACE's capability under larger spatial and data volume settings. We apply spatial translation operations to the original point cloud to generate a larger-scale point cloud dataset covering an expanded spatial region. Within this extended region, we generate new trajectories and mesh objects following the same spatial semantics. The resulting dataset contains `21b` point cloud points, `4m` trajectories (average `550` nodes), and `8.4k` mesh objects (average `160k` vertices and `116k` faces).
- *Random_EX* is a fully synthetic dataset generated to evaluate TRACE's robustness under randomly distributed and unstructured spatial data. We define a spatial boundary and uniformly sample points within it to form a point cloud. The trajectory and mesh data are then randomly generated within the same region, ensuring that there are no inherent spatial patterns or real-world structures. The dataset consists of `21b` point cloud points, `5m` trajectories, and `12.6k` mesh objects.

The diversity of these datasets, in terms of structure, size, and spatial distribution, allows a comprehensive evaluation of TRACE's performance under both real-world and synthetic conditions.

Baselines: We compare TRACE against several commonly used spatiotemporal data management solutions to evaluate its effectiveness and efficiency:

- **PostgreSQL:** All data are stored using native PostgreSQL types (e.g., `float`, `timestamp`) and indexed using a 2D `GiST` index built on the `x` and `y` coordinates. PostgreSQL is a widely adopted open-source relational database that supports extensible indexing mechanisms. The Generalized Search Tree (GiST) is a balanced tree structure that supports spatial indexing by enabling efficient range and nearest-neighbor queries over multidimensional data.
- **Ganos:** All data are stored using data types provided by `PostGIS` in the Ganos framework, with a `3D-GiST` index constructed over the dimensions `x`, `y`, and `z`. PostGIS is a spatial extension to PostgreSQL that supports advanced geometric and geographic objects. The 3D GiST index enhances spatial querying capabilities by indexing volumetric geometry and enabling efficient access to data in three-dimensional space.

Parameters: Every measurement we report is an average of 100 queries, where 10 query cases are randomly generated according



(a) Axis-Aligned query **(b) Arbitrarily Polygon query** **(c) Buffer query**

Figure 7: Query performance vs. selectivity on different datasets

to the data distribution and each case is conducted 10 times. The default parameters include.

8.1 Index Size and Construction Time

Figure 8 shows the index construction time and the storage space occupied by indexes. TRACE occupies approximately 3× less space than PostgreSQL and 4× less than Ganos. TRACE builds the index after sampling, resulting in a smaller index size and faster construction speed. TRACE's *Oc-tree* leaves store *Kd-tree*s, and the *Oc-tree* itself contains few empty nodes. Compared to PostgreSQL that have some redundant space within their nodes, *Oc-tree* achieves higher storage utilization.

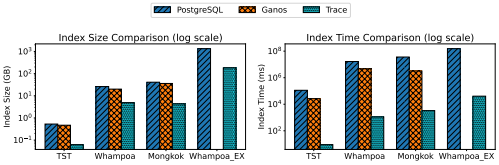


Figure 8: Index Size and Time Comparison

In terms of construction time, TRACE outperforms PostgreSQL by approximately 3 orders of magnitude and Ganos by about 4 orders of magnitude. For index construction time, TRACE first performs coarse-grained sorting of the data, then executes VC, and finally builds the index in a single phase using only VC. Therefore, the first phase is highly efficient. The second phase of inner *Oc-tree* construction only involves loading small batches of data into memory to build the *Oc-tree*, allowing for full parallel construction. Consequently, TRACE's index construction significantly outperforms both PostgreSQL and Ganos.

8.2 Query Efficiency

8.2.1 Polygonal Range Query. We tested axis-aligned and arbitrary polygon queries on four real-world datasets, examining how selectivity and result set size correlate with query latency by gradually increasing selectivity. For axis-aligned (super rectangular range) queries, we observe from Figure 7a that TRACE achieves optimal performance across all selectivity levels except for the smallest selectivity case in the TST dataset. As selectivity increases, all methods exhibit growing query latency, with latency being approximately proportional to the size of returned results. Only TRACE experiences cold-start issues during its initial query, resulting in higher latency at the 0.003% selectivity level. Compared to PostgreSQL and Ganos, TRACE's index structure maintains non-overlapping internal nodes improving index traversal efficiency. TRACE pre-packs tasks and distributes them to multiple threads during linear execution to better utilize multi-core CPU performance.

For arbitrary 3D polygonal range queries, we generate polygons by first creating 1–2 random coplanar triangles within the spatial area defined by the target selectivity, then assign a height along the z-axis based on that selectivity. As shown in Figure 7b, TRACE demonstrates approximately two orders of magnitude better query performance than Ganos, with the performance advantage becoming more pronounced at higher selectivity levels. This superiority comes from TRACE's stronger native 3D indexing and better use of multi-core CPUs, boosting 3D spatial filtering efficiency.

8.2.2 Buffer Query. Like Range Query, we tested Buffer Query under varying selectivity (with buffer size r adjusting accordingly). As shown in Figure 7c, query latency increases with selectivity for all indexes. TRACE's performance lead over the two baselines is more pronounced than in Range Query cases. On the TST dataset at 0.003% selectivity, TRACE performs similarly to PostgreSQL due to its higher *Oc-tree* resolution on small datasets, which leads to less efficient queries for small ranges.

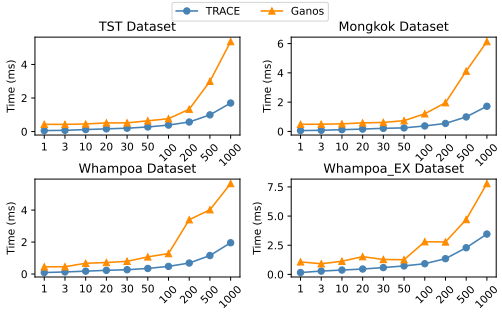


Figure 9: *kNN* Query Time vs *k*

8.2.3 kNN Query. We compared the *k*NN performance of TRACE and Ganos for querying data points on four real datasets under different *k* values. As shown in Figure 9, as *k* increases, the query latency of both TRACE and Ganos increases, but TRACE consistently demonstrates superior query performance. The reason is that both TRACE and Ganos store data at leaf nodes, but TRACE stores data in a *Kd-tree* format, while Ganos stores it unordered. During the query process, when a leaf node is loaded, TRACE can use the *Kd-tree* for pruning, whereas Ganos needs to scan the data within the

leaf node, resulting in TRACE's superior performance. The OK-tree structure provides additional performance benefits by loading entire Kd-trees into memory upon reaching Oc-tree leaf nodes during queries. This approach processes local *Kd-tree* queries entirely in memory and effectively eliminates costly random disk accesses for loading data points.

8.2.4 Similarity Query. We compared the kNN performance of TRACE and Ganos on four real datasets under different k values. The results are shown in Figure 10. As k increases, the query latency of both TRACE and Ganos increases, but TRACE consistently exhibits superior query performance. The reason is that TRACE stores data in a Kd-tree format, while Ganos stores it unordered. In leaf node queries, TRACE prunes via Kd-tree while Ganos scans all data, making TRACE perform better.

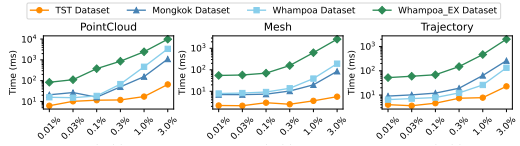


Figure 10: Similarity Query Time vs Selectivity

8.2.5 Insertion. For data updates, Figure 11 shows the time recorded when we built indexes on four different real-world datasets separately and then inserted varying amounts of data into each index. As the volume of inserted data increases, the time required for data insertion also grows. Among them, the largest dataset, *Whampoa Extension*, incurs the highest update overhead because updating the same amount of data in a larger index results in deeper tree index layers, requiring modifications to more nodes—hence the greater cost. **??** presents the time required for updating a single file with varying numbers of data files during the indexing phase on a synthetic dataset. Each file in the synthetic dataset contains exactly 10^6 data points. The experimental results demonstrate that larger initial indexing volumes correlate with marginally faster update operations. However, the update time does not scale linearly due to occasional cold starts when loading database contents into memory.

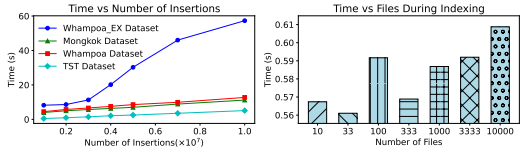
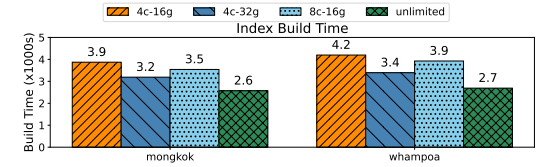


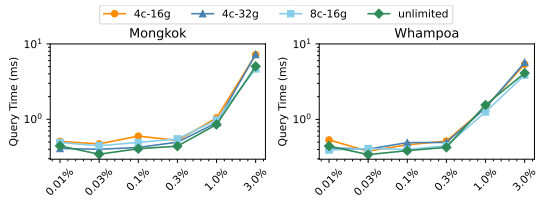
Figure 11: Insertion Time

8.3 Evaluation in Various Environment

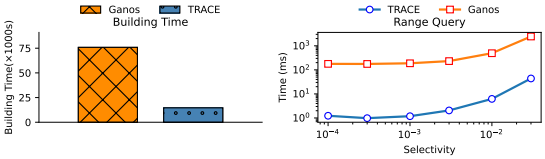
Figure 12 presents the performance of TRACE across different hardware environments, as well as a comparison between TRACE and Ganos under unrestricted hardware resource conditions. For building time result in Figure 12a, with 32GB memory, TRACE performs significantly better than with 16GB. At 16GB, TRACE consumes nearly all available memory, leaving the database with an insufficient buffer pool. The 32GB configuration provides enough buffer space for concurrent transactions. Additionally, at 16GB memory, 8 CPUs outperform 4 CPUs because TRACE's concurrent design efficiently utilizes multiple CPU cores. For range query performance shown in Figure 12b, the performance difference for range queries is relatively small under different hardware conditions. When executing individual tasks concurrently, since each query has a relatively small workload, TRACE only gains a slight performance improvement with larger memory and more CPU cores. We also conducted a performance comparison of index on the randomly generated Random_EX dataset without limiting CPU or memory resources. As shown in the Figure 12c, TRACE achieved significant performance improvements over Ganos in both the index construction and query phases. This is because TRACE's hierarchical index construction strategy allows for fully parallel building of local indexes. During the query phase, TRACE can flexibly assign tasks to different threads, resulting in overall performance optimization.



(a) Index Building Performance



(b) Polygonal Range Query Performance



(c) Unlimited Hardware Resources

Figure 12: Performance in Various Hardware Environments

9 Conclusion

This paper introduces TRACE, a unified and lightweight engine for large-scale *DST* data management and querying. With a compact *UST* data model and hierarchical dynamic indexing, TRACE efficiently executes cross-modal queries without duplicate indexes or application-level joins. Designed for scalability, parallelism, and low latency, TRACE significantly reduces memory usage and *I/O* overhead. Evaluations show the system handles large multimodal datasets, supporting both analytical and interactive tasks. TRACE is a foundational step towards a versatile backend for 3D spatiotemporal data.

References

[1] Michael Armbrust, Reynold S Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K Bradley, Xiangrui Meng, Tomer Kaftan, Michael J Franklin, Ali Ghodsi, et al. 2015. Spark sql: Relational data processing in spark. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 1383–1394.

[2] Christian Bohm, Alexey Pryakhin, and Matthias Schubert. 2006. The gauss-tree: Efficient object identification in databases of probabilistic feature vectors. In *22nd International Conference on Data Engineering (ICDE'06)*. IEEE, 9–9.

[3] Yixi Cai, Wei Xu, and Fu Zhang. 2021. ikd-tree: An incremental kd tree for robotic applications. *arXiv preprint arXiv:2102.10808* (2021).

[4] Wei Cao, Yingqiang Zhang, Xinjun Yang, Feifei Li, Sheng Wang, Qingda Hu, Xuntao Cheng, Zongzhi Chen, Zhenjun Liu, Jing Fang, et al. 2021. Polardb serverless: A cloud native database for disaggregated data centers. In *Proceedings of the 2021 International Conference on Management of Data*. 2477–2489.

[5] Su Chen, Beng Chin Ooi, Kian-Lee Tan, and Mario A Nascimento. 2008. ST2B-tree: a self-tunable spatio-temporal B+-tree index for moving objects. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 29–42.

[6] Nianbing Du, Junfeng Zhan, Ming Zhao, Daorui Xiao, and Yinchuang Xie. 2015. Spatio-temporal data index model of moving objects on fixed networks using hbase. In *2015 IEEE International Conference on Computational Intelligence & Communication Technology*. IEEE, 247–251.

[7] Fei Gao, William Wu, Wenliang Gao, and Shaojie Shen. 2019. Flying on point clouds: Online trajectory generation and autonomous navigation for quadrotors in cluttered environments. *Journal of Field Robotics* 36, 4 (2019), 710–733.

[8] Stefan Hagedorn, Philipp Gotze, and Kai-Uwe Sattler. 2017. The STARK framework for spatio-temporal data analytics on spark. In *Datenbanksysteme für Business, Technologie und Web (BTW 2017)*. Gesellschaft für Informatik, Bonn, 123–142.

[9] Ali Khaloo and David Lattanzi. 2017. Robust normal estimation and region growing segmentation of infrastructure 3D point cloud models. *Advanced engineering informatics* 34 (2017), 1–16.

[10] Alex H Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. 2019. Pointpillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 12697–12705.

[11] Yiyi Liao, Simon Donne, and Andreas Geiger. 2018. Deep marching cubes: Learning explicit surface representations. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2916–2925.

[12] Seng Poh Lim and Habibollah Haron. 2014. Surface reconstruction techniques: a review. *Artificial Intelligence Review* 42 (2014), 59–78.

[13] Timothy S Newman and Hong Yi. 2006. A survey of the marching cubes algorithm. *Computers & Graphics* 30, 5 (2006), 854–879.

[14] Regina Obe and Leo Hsu. 2021. *PostGIS in action*. Simon and Schuster.

[15] Dieter Pfoser, Christian S Jensen, Yannis Theodoridis, et al. 2000. Novel approaches to the indexing of moving object trajectories.. In *VLDB*, Vol. 2000. Citeseer, 395–406.

[16] Sayan Ranu, Padmanabhan Deepak, Aditya D Telang, Prasad Deshpande, and Sriram Raghavan. 2015. Indexing and matching trajectories under inconsistent sampling rates. In *2015 IEEE 31st International conference on data engineering*. IEEE, 999–1010.

[17] Lorenzo Rieg, Volker Wichmann, Martin Rutzinger, Rudolf Sailer, Thomas Geist, and Johann Stötter. 2014. Data infrastructure for multitemporal airborne LiDAR point cloud analysis–Examples from physical geography in high mountain environments. *Computers, Environment and Urban Systems* 45 (2014), 137–146.

[18] Simonas Šaltenis, Christian S Jensen, Scott T Leutenegger, and Mario A Lopez. 2000. Indexing the positions of continuously moving objects. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*. 331–342.

[19] Shaoshuai Shi, Chaoxu Guo, Li Jiang, Zhe Wang, Jianping Shi, Xiaogang Wang, and Hongsheng Li. 2020. Pv-rcnn: Point-voxel feature set abstraction for 3d object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10529–10538.

[20] Deepika Shukla, Chirag Shivnani, and Darshit Shah. 2016. Comparing oracle spatial and postgres PostGIS. *IJCSC* 7, 2 (2016), 95–100.

[21] Tianyun Su, Wen Wang, Haixing Liu, Zhendong Liu, Xinfang Li, Zhen Jia, Lin Zhou, Zhuanling Song, Ming Ding, and Aiju Cui. 2022. An adaptive and rapid 3D Delaunay triangulation for randomly distributed point cloud data. *The Visual Computer* (2022), 1–25.

[22] Yufei Tao and Dimitris Papadias. 2002. Time-parameterized queries in spatio-temporal databases. In *Proceedings of the 2002 ACM SIGMOD international conference on Management of data*. 334–345.

[23] David Wooden, Matthew Malchano, Kevin Blankespoor, Andrew Howardy, Alfred A Rizzi, and Marc Raibert. 2010. Autonomous navigation for BigDog. In *2010 IEEE international conference on robotics and automation*. Ieee, 4736–4741.

[24] Jiong Xie, Zhen Chen, Jianwei Liu, Fang Wang, Feifei Li, Zhida Chen, Yinpei Liu, Songlu Cai, Zhenhua Fan, Fei Xiao, et al. 2022. Ganos: a multidimensional, dynamic, and scene-oriented cloud-native spatial database engine. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3483–3495.

[25] Xiaopeng Xiong and Walid G Aref. 2006. R-trees with update memos. In *22nd International Conference on Data Engineering (ICDE'06)*. IEEE, 22–22.

[26] Yusheng Xu, Xiaohua Tong, and Uwe Stilla. 2021. Voxel-based representation of 3D point clouds: Methods, applications, and its potential use in the construction industry. *Automation in Construction* 126 (2021), 103675.

[27] Zetong Yang, Yanan Sun, Shu Liu, and Jiaya Jia. 2020. 3dssd: Point-based 3d single stage object detector. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11040–11048.

[28] Tianwei Yin, Xingyi Zhou, and Philipp Krahenbuhl. 2021. Center-based 3d object detection and tracking. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11784–11793.

[29] Yu Zheng. 2015. Trajectory data mining: an overview. *ACM Transactions on Intelligent Systems and Technology (TIST)* 6, 3 (2015), 1–41.